

Data Types

Amir Jamalian



Gisma
University
of Applied
Sciences





Data Type and Operations

- Every value has a type
 - The built-in type function returns the type of the result of any expression
 - `type(3)`
 - `type("Hello")`
- The type of an item of data (such as a string) determines what it is legal to do with the data and what the effect of various actions will be
 - E.g., the effect of using a `+` operator depends on the types involved
 - It adds numbers but concatenates strings
 - `3 + 4`
 - `"Hello " + "World"`



Dynamic Typing

- Python has dynamic typing
 - The type of the data held by a variable can dynamically change as the program executes
 - `x = 1`
 - `x = "Hello"`
- It is not the approach used by many statically typed programming languages
 - E.g., Java and C#
- Both approaches have their pros and cons
 - The flexibility of Python's variables are one of its major advantages



Data Types in Python

- Numbers

- Int
- Float

- String

- Boolean

- None

- Collection Types

- Tuple
- List
- Set
- Dictionary



- Computers are designed to perform numerical calculations
- Python distinguishes between two different types of numbers
 - Integers are called int values
 - 3, -5, 0
 - Real numbers are called float values
 - 3.5, -4.333, 0.0
- Converting to int or float
 - `int("3")`
 - `float("3.4")`



String

- Much of the world's data is text
- A piece of text represented in a computer is called a string
 - A string is a series, or sequence, of characters in order
 - Single and double quotes can both be used to create strings
 - 'hi' and "hi" are identical expressions
 - It is useful if your string needs to contain one of the other type of string delimiters
 - Triple quotes allow multiline strings
 - `s = """This is a multiline string."""`
 - We can also define an empty string
 - `s = ""`



Some String Operations

- String concatenation
 - `s = "Hello " + "World"`
- Getting the length of a string
 - `len(s)`
- Accessing a character
 - Strings are indexed from zero
 - `s[3]`
 - `s[-2]`
- Accessing a subset of characters
 - `s[2:5]`
 - `s[:5]`
 - `s[5:]`
- Repeating strings
 - `"Hi" * 10`
- Converting to string
 - `str(3)`



String Methods

- Methods are functions that operate on objects
 - We will learn more about methods later
- These methods are called by placing a dot after the (string) object, then calling the function
 - `"loud".upper()`
 - `"Hello".replace("e", "a")`



Some String Methods

- Splitting strings
 - `s.split(" ")`
- Counting in strings
 - `s.count("a")`
- Replacing strings
 - `s.replace("x", "y")`
- Finding sub strings
 - `s.find("hi")`
- Formatting string
 - `"The name is {}".format("John")`
 - `f"The name is {John}."`



Mutable and Immutable Types

- Python has two kinds of built-in or user-defined types
- Mutable types allow in-place modification of the object's content
 - Such as lists
 - `my_list = [1, 2, 3]`
 - `my_list[0] = 4` # The new list is [4, 2, 3]
 - They have mutating methods like `list.append()` that can modify the object in place
- Immutable types do not allow in-place modification of the object's content
 - Such as strings
 - `my_string = "123"`
 - `my_string[0] = "4"` # It generates an error
 - These types provide no method for changing their content in place
 - We have to create another variable to store the modified object





Immutability

- Strings are immutable (not changable)
 - Once a string object has been created, it cannot be changed in place
- If you try to change a string, you will create a new string
 - You will not affect the original string in anyway
 - Remember to store the result
 - `s = s.replace("x", "y")`



- A Boolean type can only be one of True or False
 - The Boolean type is actually a sub type of integer
 - It is easy to translate between these two types using `int()` and `bool()` functions
 - Python is quite in representing True and False
 - 0, "" (empty strings), None equate to False
 - Non-zero, non-empty strings, any object equate to True
 - It is recommend to just use True and False
 - It is often a cleaner and safer approach
- Boolean values most often arise from comparison operators
 - The comparison `3 > 2` generates True



None

- Python has a special type, the `NoneType`, with a single value, `None`
 - This is used to represent null values or nothingness
 - It is not the same as `False`, an empty string, or `0`
 - it is a non-value
- It can be used to initiate a variable that we do not have an initial value for
 - `winner = None`
 - You can then test for the presence of `None` using “`is`” and “`is not`” operators
 - `print(winner is None)`



Identity Operators

- Identity operators are used to compare the objects

- Not if they are equal, but if they are actually the same object, with the same memory location

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y



Collection Types

- A collection is a single object representing a group of objects
 - It is also referred to as containers
- These collection classes are often used as the basis
 - For more complex or application specific data structures and data types



Collection Types in Python

- **Tuples**

- Tuple represents a collection of objects that are ordered and immutable
- Tuples allow duplicate members and are indexed

- **Lists**

- Lists hold a collection of objects that are ordered and mutable
- They are indexed and allow duplicate members

- **Sets**

- Sets are a collection that is unordered and unindexed
- They are mutable but do not allow duplicate values to be held

- **Dictionary**

- A dictionary is an unordered collection that is indexed by a key which references a value
- No duplicate keys are allowed
- Duplicate values are allowed
- Dictionaries are mutable containers

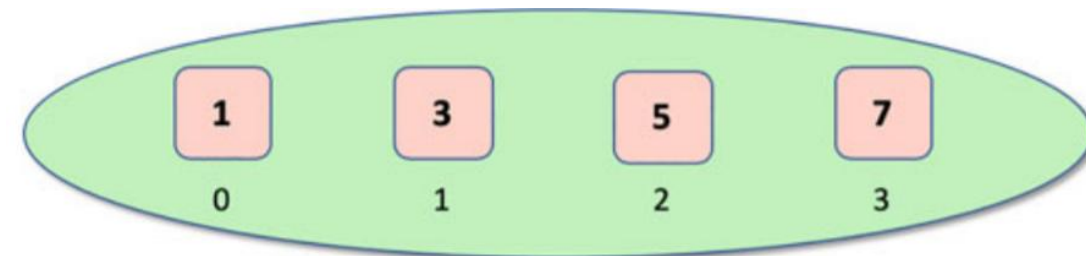


Tuples

[Hunt 2019]

17

- Tuples are an immutable ordered collection of objects
 - It is not possible to add or remove elements from the tuple once it has been created
- Tuples are defined using parentheses
 - `t = (1, 3, 5, 7)`
 - This tuple contains exactly 4 elements
 - The first element (the integer 1) has the index 0 and the last element (the integer 7) has the index 3
- Tuples can also contain a mixture of different types
 - `t = (1, "Hello", True, (1, 3, 5))`





Some Tuple Operations and Methods

- Converting to tuple
 - `tuple([1, 2, 3])`
- Accessing elements of a tuple
 - `t[3]`
- Accessing a subset of elements of a tuple
 - `t[3:5]`
- Getting the length of a tuple
 - `len(t)`
- Counting an element in a tuple
 - `t.count("apple")`
- Finding the first index of a value in a tuple
 - `t.index("apple")`



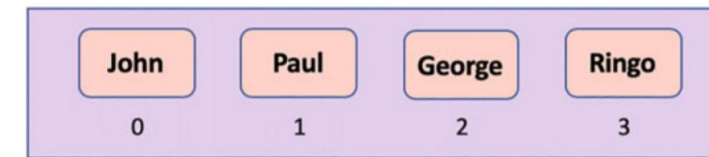
Lists

[Hunt 2019]

19

- Lists are mutable ordered containers of other objects

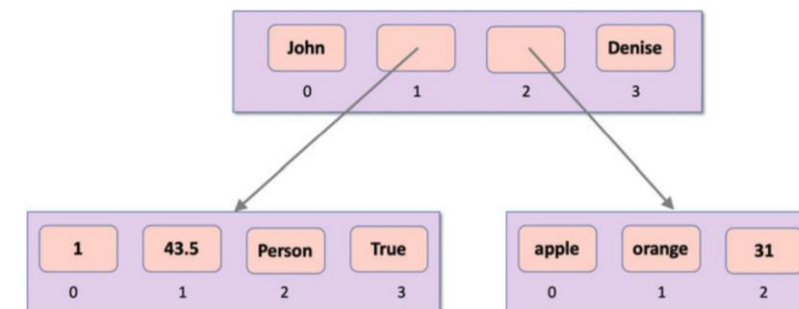
- They support all the features of the tuple
- Furthermore, they allow to add, remove, and modify elements



- Lists are created using square brackets

- `l = ["John", "Paul", "George", "Ringo"]`

- Similar to tuples, lists can contain a mixture of data types with a nested structure





Some List Operations and Methods

- Converting to list
 - `list((1, 2, 3))`
- Adding elements to a list
 - `l.append(4)`
 - It changes the actual list
- Adding a list to a list
 - `l.extend([5, 6])`
 - `l += [5, 6]`
- Inserting elements into a list
 - `l.insert(2, 1000)`
- Removing elements by value from a list
 - `l.remove(4)`
- Removing elements by index from a list
 - `l.pop(2)`
- Reversing elements in a list
 - `l.reverse()`
- Sorting elements in a list
 - `l.sort()`



Sets

- Set is an unordered (un indexed) collection of immutable objects that does not allow duplicates
- A Set is defined using curly brackets
 - $s = \{1, 2, 3\}$
- Sets can hold any immutable object
 - We cannot nest lists or other sets within a set as these are not immutable types



Some Set Operations and Methods

- Converting to set
 - `set([1, 1, 2, 3])`
- Checking for presence of an element
 - `print(1 in s)`
- Adding an item to a set
 - `s.add(4)`
- Adding multiple items to a set
 - `s.update([5, 6, 7])`
- Removing an Item from set
 - `s.remove(5)`



Some Set Operations and Methods

[Hunt 2019]

23

- Union

- `s1.union(s2)`
- `s1 | s2`

- Intersection

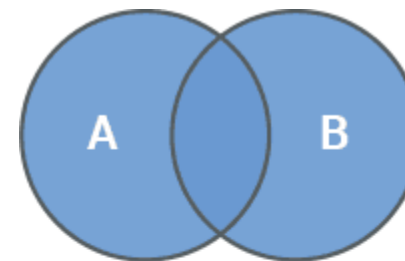
- `s1.intersection(s2)`
- `s1 & s2`

- Difference

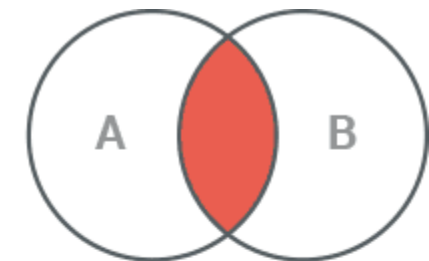
- `s1.difference(s2)`
- `s1 - s2`

- Symmetric difference

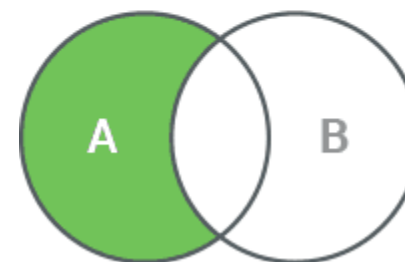
- `s1.symmetric_difference(s2)`
- `s1 ^ s2`



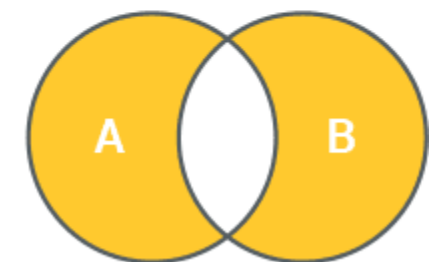
Union



Intersection



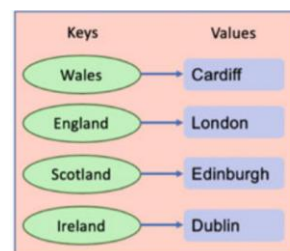
Difference



Symmetric Difference



- A Dictionary is a set of associations between a key and a value
 - The dictionary is unordered, changeable (mutable) and indexed
 - The keys must be unique, but the values do not need to be unique
 - Keys must be immutable
- A Dictionary is created using curly brackets ('{}') where each entry is a key:value pair
 - `d = {'Wales': 'Cardiff', 'England': 'London', 'Scotland': 'Edinburgh', 'Northern Ireland': 'Belfast', 'Ireland': 'Dublin'}`
 - The `dict()` function can be used to create a new dictionary object from an iterable or a sequence of key:value pairs
- A dictionary can be nested
 - The value itself can be a container





Some Dictionary Operations and Methods

- Accessing items via keys
 - `d['Wales']`
- Adding a new entry
 - `d['France'] = 'Paris'`
- Changing a keys value
 - `d['Wales'] = 'Swansea'`
- Removing an entry
 - `d.pop("Wales")`
- Checking a key membership
 - `print("Wales" in d)`
- Accessing to keys, values, or key-value pairs
 - `d.keys()`
 - `d.values()`
 - `d.items()`



Membership Operators

- Membership operators are used to test if a sequence is presented in an object

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y



Where to Use What?

- The time complexity of different operations on different collection types is different
 - For example, the time complexity of finding an element in a List is $O(n)$ while it is $O(1)$ in a dictionary
- Be careful to choose the right collection type
 - Check out “data structures and algorithms” textbooks for more information